

Experiment 25: Monkey and Banana Problem Using Prolog

Aim

To implement the **Monkey and Banana problem using Prolog** and demonstrate how logical reasoning and state-space representation can be used to find a solution.

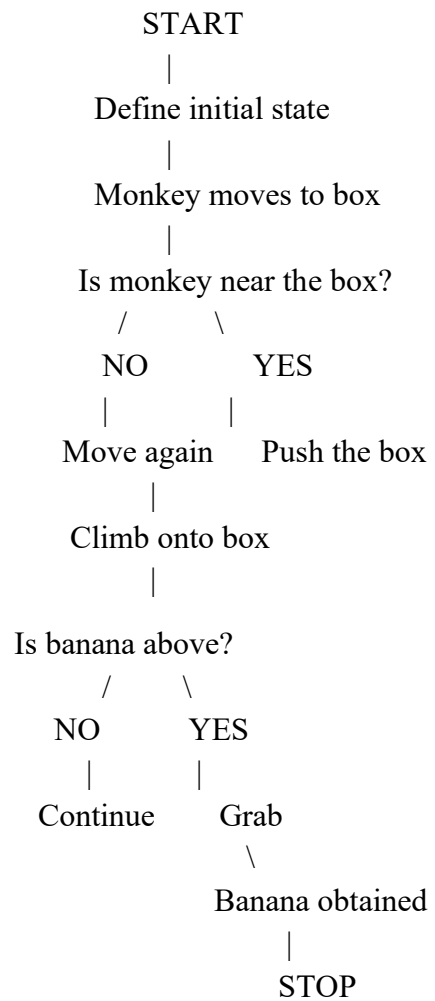
Objective

1. To understand the Monkey and Banana problem in Artificial Intelligence.
2. To represent different states of the monkey, box, and banana in Prolog.
3. To implement actions such as moving, pushing, climbing, and grabbing.
4. To use Prolog rules to determine whether the monkey can obtain the banana.

Algorithm

1. Start the program.
2. Define the initial state of the monkey, box, and banana.
3. Define the move operation to move the monkey.
4. Define the push operation to move the box.
5. Define the climb operation to allow the monkey to climb onto the box.
6. Define the grab operation to allow the monkey to obtain the banana.
7. Define a solve rule to find a sequence of valid actions.
8. Enter the initial state as a query.
9. Prolog applies the rules and generates the required actions.
10. Display the final state and sequence of actions.
11. Stop.

Flowchart



Prolog Code

% Monkey and Banana Problem

% Monkey can move

```
move(state(M, B, onfloor, H),  
      state(P, B, onfloor, H),  
      move) :-  
    M \= P.
```

% Monkey can push the box

```
push(state(M, M, onfloor, H),  
      state(P, P, onfloor, H),  
      push_box) :-  
    M \= P.
```

% Monkey can climb onto the box

```
climb(state(M, M, onfloor, H),  
      state(M, M, onbox, H),  
      climb).
```

% Monkey can grab the banana

```
grab(state(M, M, onbox, M),  
      state(M, M, onbox, grabbed),  
      grab).
```

% Solution

```
solve(State, State, []) :-  
    State = state(_, _, onbox, grabbed).
```

```
solve(State, Final, [Action|Actions]) :-  
    ( move(State, Next, Action)  
    ; push(State, Next, Action)  
    ; climb(State, Next, Action)  
    ; grab(State, Next, Action)  
    ),  
    solve(Next, Final, Actions).
```

Sample Query

Use the following query:

```
?- solve(state(door, window, onfloor, center), Final, Actions).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with options: File, Settings, Tools, Debug, and Help. The main area of the window is a black terminal with white text. It displays a welcome message, a license notice, and a URL for online help. It then shows two Prolog queries being executed. The first query is a consult statement that succeeds. The second query is a grab statement that also succeeds, returning a state and a list of actions.

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

40 ?- consult(['c:/Users/User/OneDrive/Desktop/monkey.pl']).
true.

41 ?- grab(state(center, center, onbox, center), Result).
Result = state(center, center, onbox, grabbed).
```

Result

The **Monkey and Banana problem** was successfully implemented using Prolog. The program determines a sequence of actions that allows the monkey to reach the banana and grab it.

Conclusion

Thus, the experiment demonstrates the use of **Prolog for problem solving through state representation, rules, and logical inference**. The Monkey and Banana problem was solved by applying a sequence of valid actions such as moving, pushing, climbing, and grabbing.